



Mobile Application Development



Introduction

Advanced Android applications enhance user experience through:

- Visual animations
- Smooth transitions
- Multi-screen navigation

Key focus:

- Animation techniques
- Activity lifecycle and navigation



What is Animation in Android?

- Animation = visual changes in UI over time
- Makes apps:
 - Interactive
 - Engaging
 - User-friendly
- Types:
 - View Animation
 - Property Animation
 - Drawable Animation



Changing Transparency (Alpha Animation)

- Used to fade views in/out
- Property: **alpha (0.0 to 1.0)**
 - 0 → fully transparent
 - 1 → fully visible

Example:

```
view.animate().alpha(0f).setDuration(1000);
```

Uses:

- Fade-in buttons
- Hide/show elements smoothly



Changing Size (Scaling Animation)

- Modifies width and height of a view
- Properties:
 - `scaleX`
 - `scaleY`

Example:

```
view.animate().scaleX(2f).scaleY(2f).setDuration(1000);
```

Uses:

- Zoom effects
- Emphasizing UI elements



Movement Animation (Translation)

- Moves an object from one position to another
- Properties:
 - `translationX`
 - `translationY`

Example:

```
view.animate().translationX(200f).setDuration(1000);
```

Uses:

- Sliding menus
- Moving icons/buttons



Rotation Animation

- Rotates a view around its pivot point

Example:

```
view.animate().rotation(360f).setDuration(1000);
```

Uses:

- Loading animations
- Interactive UI elements



Combining Animations

- Multiple animations can be applied together

Example:

```
view.animate()  
  .alpha(0.5f)  
  .scaleX(1.5f)  
  .rotation(180f)  
  .setDuration(1000);
```

- Creates smooth and complex effects



What is an Activity?

Activity = single screen in Android app

Example:

- Login screen
- Home screen

Managed by Android lifecycle



Activity Lifecycle (Brief)

Key methods:

- `onCreate()`
- `onStart()`
- `onResume()`
- `onPause()`
- `onStop()`
- `onDestroy()`

Helps manage app behavior



Creating a Second Activity

Steps:

1. Create new Activity class
2. Define layout XML
3. Register in AndroidManifest.xml

Example:

```
public class SecondActivity extends AppCompatActivity {  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_second);  
    }  
}
```



Invoking (Calling) Second Activity

- Use **Intent** to navigate

Example:

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);  
startActivity(intent);
```

- Enables screen switching



Passing Data Between Activities

- Use Intent extras

Example:

```
intent.putExtra("key", "value");
```

Receiving:

```
String data = getIntent().getStringExtra("key");
```



Importance of Animations & Activities

Enhances User Experience (UX)

Animations make interactions smooth and visually appealing, while activities organize content into clear screens.

Improves Navigation Flow

Activities enable structured screen transitions, and animations make those transitions intuitive and seamless.

Increases User Engagement

Dynamic animations capture attention and keep users interested in the application.

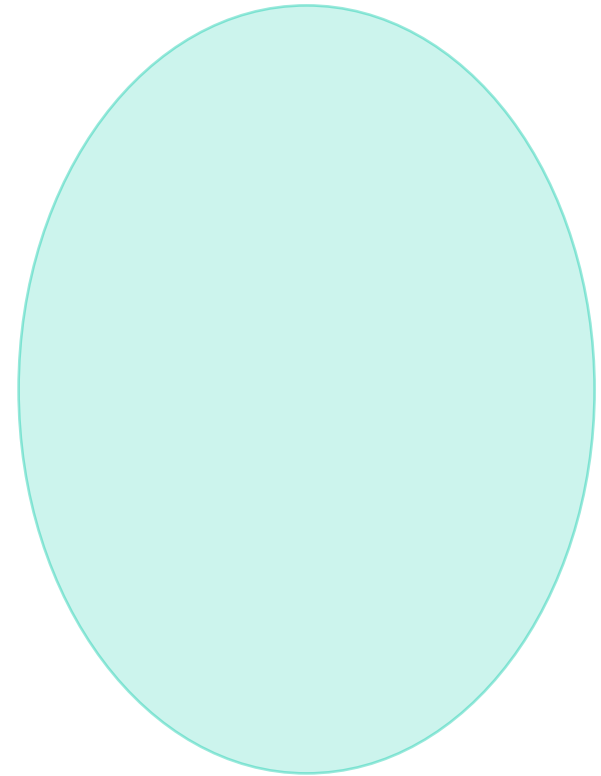
Provides Visual Feedback

Animations help users understand actions (e.g., button press, loading), improving usability.

Supports Modular App Design

Activities divide the app into manageable components, making development and maintenance easier.

ADVANCED APPLICATIONS



Transparency • Size • Movement • Rotation • Multiple Activities

Mobile Application Development

TOPICS COVERED

01 What is Android Animation?

Overview and types of animation in Android

02 Changing Transparency & Size

Alpha animations and ScaleAnimation

03 Movement & Rotation

TranslateAnimation and RotateAnimation

04 Creating a Second Activity

Intents, manifest setup, and layout

05 Invoking the Second Activity

startActivity(), putExtra(), data passing

01 INTRODUCTION

Android Animation allows developers to add visual motion effects to UI components – improving user experience and making interfaces feel alive.

View Animation

Tween (translate, rotate, scale, alpha)

Drawable Animation

Frame-by-frame animation

Property Animation

ValueAnimator / ObjectAnimator

XML Animation Resource

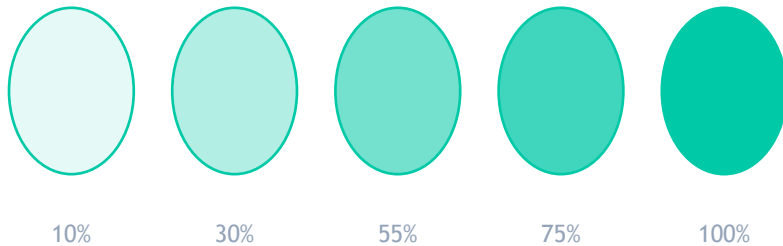
```
<!-- res/anim/my_animation.xml -->
<set xmlns:android=
    "http://schemas.android.com/apk/res/android">

    <alpha
        android:fromAlpha="0.0"
        android:toAlpha="1.0"
        android:duration="1000"/>

    <scale
        android:fromXScale="0.5"
        android:toXScale="1.0"
        android:duration="800"/>

</set>
```

02 ANIMATION



fromAlpha

toAlpha

alpha_animation.xml

```
<alpha
    android:fromAlpha="0.0"
    android:toAlpha="1.0"
    android:duration="1500"
    android:repeatCount="1"
    android:repeatMode="reverse"/>
```

MainActivity.java

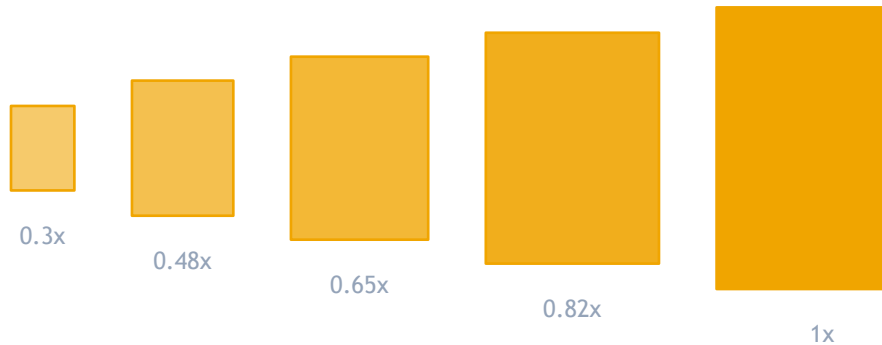
```
ImageView img = findViewById(R.id.myImage);

// Load animation from XML
Animation alphaAnim = AnimationUtils
    .loadAnimation(this,
        R.anim.alpha_animation);

// Start the animation
img.startAnimation(alphaAnim);

// Or use programmatically
AlphaAnimation anim =
    new AlphaAnimation(0.0f, 1.0f);
anim.setDuration(1500);
img.startAnimation(anim);
```

02 ANIMATION



scale_animation.xml

```
<scale
    android:fromXScale="0.5"
    android:toXScale="1.5"
    android:fromYScale="0.5"
    android:toYScale="1.5"
    android:pivotX="50%"
    android:pivotY="50%"
    android:duration="1000"/>
```

Java - Programmatic Scale

```
// ScaleAnimation(fromX, toX, fromY, toY,
//   pivotXType, pivotXVal,
//   pivotYType, pivotYVal)
ScaleAnimation scale =
    new ScaleAnimation(
        0.5f, 1.5f,
        0.5f, 1.5f,
        Animation.RELATIVE_TO_SELF, 0.5f,
        Animation.RELATIVE_TO_SELF, 0.5f
    );
scale.setDuration(1000);
scale.setFillAfter(true);
view.startAnimation(scale);
```

03 ANIMATION



fromX/fromY —————> toX/toY

translate_animation.xml

```
<translate
    android:fromXDelta="0"
    android:toXDelta="200"
    android:fromYDelta="0"
    android:toYDelta="-150"
    android:duration="1200"
    android:fillAfter="true"/>
```

Java - TranslateAnimation

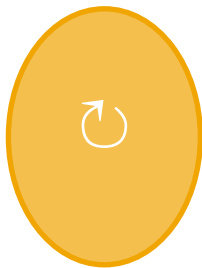
```
// Pixels or percentage
// TranslateAnimation(fromX, toX, fromY, toY)
TranslateAnimation move =
    new TranslateAnimation(0, 200, 0, -150);

move.setDuration(1200);
move.setFillAfter(true);
move.setInterpolator(new
    AccelerateDecelerateInterpolator());

view.startAnimation(move);

// Relative to parent
new TranslateAnimation(
    Animation.RELATIVE_TO_PARENT, 0f,
    Animation.RELATIVE_TO_PARENT, 1.0f,
    0, 0, 0, 0);
```

03 ANIMATION



0° → 360°

Pivot Point: center (50%, 50%)

rotate_animation.xml

```
<rotate
    android:fromDegrees="0"
    android:toDegrees="360"
    android:pivotX="50%"
    android:pivotY="50%"
    android:duration="1000"
    android:repeatCount="infinite"/>
```

Java - RotateAnimation

```
// RotateAnimation(fromDeg, toDeg,
//   pivotXType, pivotX,
//   pivotYType, pivotY)
RotateAnimation rotate =
    new RotateAnimation(
        0, 360,
        Animation.RELATIVE_TO_SELF, 0.5f,
        Animation.RELATIVE_TO_SELF, 0.5f
    );

rotate.setDuration(1000);
rotate.setRepeatCount(
    Animation.INFINITE);
rotate.setInterpolator(new
    LinearInterpolator());

imageView.startAnimation(rotate);
```

03 ANIMATION

duration

Total time for animation (ms)

startOffset

Delay before animation starts

fillAfter

Keep final state after animation

repeatCount

Number of repeats (INFINITE = -1)

interpolator

Speed curve (Accelerate, Linear, Bounce...)

Combine in Java

```
AnimationSet animSet =  
    new AnimationSet(true);
```

```
AlphaAnimation alpha =  
    new AlphaAnimation(0f, 1f);  
alpha.setDuration(800);
```

```
ScaleAnimation scale =  
    new ScaleAnimation(0f, 1f, 0f, 1f,  
        Animation.RELATIVE_TO_SELF, 0.5f,  
        Animation.RELATIVE_TO_SELF, 0.5f);  
scale.setDuration(800);
```

```
RotateAnimation rotate =  
    new RotateAnimation(0, 360, ...);  
rotate.setDuration(1000);
```

```
animSet.addAnimation(alpha);  
animSet.addAnimation(scale);  
animSet.addAnimation(rotate);  
view.startAnimation(animSet);
```

04 ACTIVITIES

1

Create Java Class

Right-click package → New → Java Class
Extend AppCompatActivity

2

Add Layout XML

Create activity_second.xml in res/layout/
Design your UI components

3

setContentView()

In onCreate(), call:
`setContentView(R.layout.activity_second)`

4

Register in Manifest

Add <activity> tag in AndroidManifest.xml

04 ACTIVITIES

AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapplication">

    <application
        android:allowBackup="true"
        android:label="@string/app_name">

        <!-- Main Activity -->
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>

        <!-- Second Activity - MUST be registered here -->
        <activity android:name=".SecondActivity"
            android:label="Second Screen"/>

    </application>
</manifest>
```


05 ACTIVITIES



Simple Intent - MainActivity.java

```
Button btn = findViewById(R.id.btnLaunch);

btn.setOnClickListener(v -> {
    // Create Intent
    Intent intent = new Intent(
        MainActivity.this,
        SecondActivity.class
    );

    // Start the Activity
    startActivity(intent);
});
```

Passing Data with putExtra()

```
Intent intent = new Intent(
    this, SecondActivity.class);

// Send data
intent.putExtra("username", "Alice");
intent.putExtra("score", 95);

startActivity(intent);

// In SecondActivity - receive data
Intent received = getIntent();
String name =
    received.getStringExtra("username");
int score =
    received.getIntExtra("score", 0);
```

05 ACTIVITIES

SecondActivity.java

```
public class SecondActivity
    extends AppCompatActivity {

    TextView tvName, tvScore;

    @Override
    protected void onCreate(
        Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(
            R.layout.activity_second);

        tvName = findViewById(R.id.tvName);
        tvScore = findViewById(R.id.tvScore);

        // Get Intent data
        Intent i = getIntent();
        String name = i.getStringExtra(
            "username");
        int score = i.getIntExtra("score",0);

        tvName.setText("User: " + name);
        tvScore.setText("Score: " + score);
    }
}
```

activity_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android
        .com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="20sp"
        android:text="User:"/>

    <TextView
        android:id="@+id/tvScore"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="20sp"
        android:text="Score:"/>

</LinearLayout>
```

SUMMARY



Alpha Animation

Controls transparency from 0.0 (invisible) to 1.0 (opaque)



Scale Animation

Resizes view from pivot point using X and Y scale factors



Translate Animation

Moves view by fromX/Y → toX/Y pixel or percentage offsets



Rotate Animation

Rotates view around a pivot from fromDeg to toDeg



Second Activity

Extend AppCompatActivity, register in Manifest, design layout



Invoking Activity

Use Intent + startActivity(); pass data via putExtra()

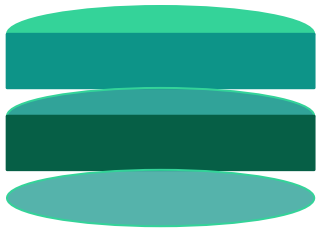


Conclusion

- 1. Animations enhance interactivity**
They make the application visually engaging and responsive to user actions.
- 2. Activities enable multi-screen functionality**
They structure the app into different screens for better organization.
- 3. Smooth transitions improve usability**
Combining animations with activities creates seamless user navigation.
- 4. Better user engagement and satisfaction**
Attractive UI and clear navigation keep users interested and satisfied.
- 5. Essential for modern Android development**
Both concepts are fundamental for building professional and user-friendly apps.



Part 2



Content Providers

Persistent Storage in Android

SQLite

ContentProvider

Transactions

FileStorage

Search



Course Outline

01 Introduction to SQLite & Persistent Storage

02 Opening and Creating Databases

03 Querying, Adding, Updating & Removing Rows

04 Creating Content Providers

05 Transactions in Android

06 Storing & Accessing Files in Content Providers

07 Adding, Deleting & Updating Content

08 Searching the Content Provider

01

Introduction to SQLite

Persistent Storage in Android Applications



What is SQLite?

SQLite is a lightweight, self-contained relational database engine embedded directly in Android.

No separate server process — the database lives as a single .db file on device storage.

Android SDK provides full SQLite support through the `android.database.sqlite` package.

Ideal for structured local data: user profiles, app content, cached API responses.

ACID-compliant — Atomicity, Consistency, Isolation, Durability — ensuring data integrity.

Supports standard SQL: CREATE, SELECT, INSERT, UPDATE, DELETE statements.

Data persists across app sessions and device restarts — perfect for offline-first apps.

Key Concepts

```
// SQLite version check
Log.d("DB", "SQLite version: " +
    android.database.sqlite
        .SQLiteDatabase
        .getVersion())

// Data is stored at:
// /data/data/<package>/
//   databases/<db-name>.db

// Core Classes:
// SQLiteOpenHelper — manages DB
// SQLiteDatabase   — executes SQL
// Cursor           — result set
```

02

Opening and Creating Databases

SQLiteOpenHelper – Lifecycle Management



SQLiteOpenHelper

SQLiteOpenHelper manages database creation and version management automatically.

Subclass it and override onCreate() to define your tables on first app launch.

Override onUpgrade() to handle schema migrations between app version updates.

Call getWritableDatabase() for read/write access; getReadableDatabase() for read-only.

The helper caches the connection — avoid calling getWritableDatabase() in loops.

Close the database with close() when done, or let the helper manage lifecycle.

Use singleton pattern for the helper to prevent multiple concurrent open connections.

SQLiteOpenHelper

```
class MyDbHelper(context: Context)
    : SQLiteOpenHelper(context,
        "myapp.db", null, 1) {

    override fun onCreate(db: SQLiteDatabase) {
        db.execSQL("""
            CREATE TABLE users (
                _id    INTEGER PRIMARY KEY,
                name   TEXT NOT NULL,
                email  TEXT UNIQUE,
                age    INTEGER
            )
        """).trimIndent()
    }

    override fun onUpgrade(
        db: SQLiteDatabase,
        oldV: Int, newV: Int) {
        db.execSQL("DROP TABLE IF EXISTS users")
        onCreate(db)
    }
}

// Usage
val db = MyDbHelper(context)
    .writableDatabase
```

03

Querying, Adding, Updating & Removing Rows

CRUD Operations with SQLiteDatabase



Adding & Querying Rows

Use ContentValues to package column name-value pairs for insert and update operations.

db.insert(table, nullColumnHack, values) returns the new row's ID, or -1 on failure.

db.query() provides a structured, injection-safe alternative to raw SQL SELECT.

Always use ? placeholders for arguments – never concatenate user input into SQL.

query() returns a Cursor – iterate with moveToFirst() and moveToNext().

Always close Cursors after use to avoid memory and connection leaks.

Use.rawQuery() for complex SQL with JOINs or subqueries.

Insert & Query

```
// INSERT a new row
val values = ContentValues().apply {
    put("name", "Alice")
    put("email", "alice@mail.com")
    put("age", 28)
}
val newId = db.insert("users", null, values)

// SELECT with query()
val cursor = db.query(
    "users",                // table
    arrayOf("_id", "name", "age"), // cols
    "age > ?",              // where
    arrayOf("18"),          // args
    null, null, "name ASC"  // orderBy
)
cursor.use {
    while (it.moveToNext()) {
        val name = it.getString(
            it.getColumnIndexOrThrow("name"))
        Log.d("DB", name)
    }
}
```

Updating and Removing Rows

`db.update(table, values, where, args)` – returns the count of rows updated.

`db.delete(table, where, args)` – returns the number of rows deleted.

Always supply a WHERE clause to avoid unintended full-table updates or deletes.

Pass null as WHERE clause only when intentionally modifying all rows.

Use `db.execSQL()` for DDL statements like CREATE, DROP, ALTER TABLE.

Prefer parameterized methods over `execSQL` for DML – safer against SQL injection.

Check return values to confirm operations succeeded before notifying the UI.

Update & Delete

```
// UPDATE a row
val updates = ContentValues().apply {
    put("age", 29)
}
val rows = db.update(
    "users",
    updates,
    "_id = ?",
    arrayOf("1")
)
Log.d("DB", "Updated $rows row(s)")

// DELETE specific rows
val deleted = db.delete(
    "users",
    "age < ?",
    arrayOf("18")
)
Log.d("DB", "Deleted $deleted row(s)")

// DELETE ALL rows
db.delete("users", null, null)

// DDL via execSQL
db.execSQL(
    "ALTER TABLE users ADD COLUMN phone TEXT")
```

04

Creating Content Providers

Sharing Structured Data Across Android Apps



What is a Content Provider?

A Content Provider exposes structured data to other apps via a standard URI interface.

Uses Android IPC — apps communicate through URIs like `content://com.example/users`.

Required to share data with other apps, sync adapters, or the Android Search Framework.

Can be backed by any data source: SQLite, files, network, or in-memory stores.

Access is controlled via `android:permission` attributes in the `AndroidManifest`.

CRUD operations are performed via `ContentResolver`, not directly on the database.

Built-in providers include: `Contacts`, `MediaStore`, `Calendar`, and `Settings`.

Provider Basics

```
// AndroidManifest.xml
<provider
    android:name=".MyProvider"
    android:authorities=
        "com.example.provider"
    android:exported="true"
    android:readPermission=
        "com.example.READ"
    android:writePermission=
        "com.example.WRITE" />

// Accessing from any app:
val resolver = context.contentResolver
val uri = Uri.parse(
    "content://com.example.provider/users")

val cursor = resolver.query(
    uri, null, null, null, null)
```


Implementing a Content Provider

Extend `ContentProvider` and implement: `query`, `insert`, `update`, `delete`, and `getType`.

Define a `CONTENT_URI` constant using your authority string for external references.

Use `UriMatcher` to map URI patterns to integer codes for routing inside the provider.

The `#id` suffix pattern matches single-row URIs like `content://.../users/42`.

`getType()` returns the MIME type –
`vnd.android.cursor.dir/...` for lists,
`vnd.android.cursor.item/...` for singles.

Call

`getContext().getContentResolver().notifyChange(uri, null)` after any data mutations.

Register the provider in `AndroidManifest.xml` with the correct authority and permissions.

Provider Implementation

```
class MyProvider : ContentProvider() {
    companion object {
        const val AUTHORITY =
            "com.example.provider"
        val BASE_URI: Uri =
            Uri.parse("content://$AUTHORITY")
        val USERS_URI =
            Uri.withAppendedPath(BASE_URI, "users")
        const val USERS = 1
        const val USER_ID = 2
        val matcher = UriMatcher(
            UriMatcher.NO_MATCH).apply {
                addURI(AUTHORITY, "users", USERS)
                addURI(AUTHORITY, "users/#", USER_ID)
            }
    }

    override fun query(uri: Uri, ...) =
        when (matcher.match(uri)) {
            USERS -> db.query("users", ...)
            USER_ID -> db.query("users",
                selection = "_id=?",
                selectionArgs = arrayOf(
                    uri.lastPathSegment))
            else -> throw IllegalArgumentException()
        }
}
```

05

Transactions

Ensuring Data Integrity with Atomic Operations



Database Transactions

A transaction groups multiple operations into a single atomic unit — all succeed or all fail.

Without transactions, partial failures leave the database in an inconsistent state.

`beginTransaction()` starts the transaction;
`setTransactionSuccessful()` marks it for commit.

`endTransaction()` in a finally block ensures the transaction always closes, rolling back on failure.

Transactions dramatically improve bulk insert performance — often 100× faster or more.

Nested transactions are supported via `beginTransactionNonExclusive()`.

Use transactions for any multi-step operation where partial completion is unacceptable.

Transaction Pattern

```
val db = helper.writableDatabase
db.beginTransaction()
try {
    // Step 1: insert order
    db.insert("orders", null, order)

    // Step 2: update inventory
    val inv = ContentValues().apply {
        put("stock", newStock)
    }
    db.update("inventory", inv,
        "product_id=?",
        arrayOf(productId.toString()))

    // Step 3: write audit log
    db.insert("audit_log", null, log)

    // Mark for commit only if all ok
    db.setTransactionSuccessful()
} catch (e: Exception) {
    Log.e("DB", "Rolled back!", e)
} finally {
    db.endTransaction() // commit or rollback
}
```

06

Storing Files in a Content Provider

Binary Data and File URI Management



File Storage in Content Providers

Content Providers can serve files (images, audio, PDFs) via `openFile()` / `openAssetFile()`.

Return a `ParcelFileDescriptor` pointing to the actual file on disk from `openFile()`.

Store file paths or names in the SQLite DB; serve the binary content on demand via URL.

Use `MODE_READ_ONLY`, `MODE_WRITE_ONLY`, or `MODE_READ_WRITE` when opening files.

`FileProvider` (from Jetpack) simplifies secure file sharing without exposing real paths.

Android 7.0+ requires `FileProvider` – direct `file://` URIs throw `FileUriExposedException`.

Grant temporary URI permissions with `FLAG_GRANT_READ_URI_PERMISSION` when sharing.

File Serving

```
// In Provider – override openFile()
override fun openFile(
    uri: Uri, mode: String
): ParcelFileDescriptor? {
    val id = uri.lastPathSegment
    val path = getFilePath(id)
    val file = File(path)
    return ParcelFileDescriptor
        .open(file,
            ParcelFileDescriptor.MODE_READ_ONLY)
}

// FileProvider in Manifest:
<provider
    android:name=
        "androidx.core.content.FileProvider"
    android:authorities=
        "com.example.fileprovider"
    android:grantUriPermissions="true">
    <meta-data
        android:name=
            "android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths"/>
</provider>
```

Accessing Files from a Content Provider

Clients use `ContentResolver.openInputStream()` to read file content from a provider URI.

`ContentResolver.openOutputStream()` is used for writing files through the provider.

Use `ContentResolver.openFileDescriptor()` when you need a raw file descriptor.

`BitmapFactory.decodeStream()` can be chained directly with `openInputStream()` for images.

Always close streams in a finally block, or use Kotlin's use extension function.

Check the MIME type first with `getType()` to confirm the format before reading.

Process large files on a background thread — never block the main/UI thread.

Client-Side Access

```
// Read file from provider as InputStream
val uri = Uri.parse(
    "content://com.example/files/1")

context.contentResolver
    .openInputStream(uri)?.use { stream ->
        val bytes = stream.readBytes()
        // process bytes...
    }

// Load image directly into ImageView
context.contentResolver
    .openInputStream(uri)?.use { stream ->
        val bmp = BitmapFactory
            .decodeStream(stream)
        imageView.setImageBitmap(bmp)
    }

// Write through provider
context.contentResolver
    .openOutputStream(uri)?.use { out ->
        out.write(data)
    }
```

07

Adding, Deleting & Updating Content

ContentResolver CRUD Interface



ContentResolver: Insert, Update, Delete

ContentResolver is the client-side interface for all Content Provider operations.

resolver.insert(uri, values) returns the URI of the newly created item.

resolver.update(uri, values, where, args) returns count of rows modified.

resolver.delete(uri, where, args) returns count of rows deleted.

bulkInsert(uri, values[]) enables faster batch inserts; provider may optimise internally.

applyBatch() supports atomically applying multiple ContentProviderOperation objects.

Changes via ContentResolver automatically trigger registered ContentObservers.

ContentResolver

```
val resolver = context.contentResolver
val uri = MyProvider.USERS_URI

// INSERT
val values = ContentValues().apply {
    put("name", "Bob")
    put("email", "bob@mail.com")
}
val newUri = resolver.insert(uri, values)
// newUri = content://.../users/5

// UPDATE
val upd = ContentValues().apply {
    put("age", 30)
}
resolver.update(uri, upd, "_id=?",
    arrayOf("5"))

// DELETE
resolver.delete(uri, "_id=?",
    arrayOf("5"))

// BULK INSERT
val batch = Array(100) { i ->
    ContentValues().apply { put("val", i) }
}
resolver.bulkInsert(uri, batch)
```


08

Searching the Content Provider

Search Framework & ContentObserver



Searching Content Providers

Expose search via a `SearchRecentSuggestionsProvider` or a custom suggestion provider.

Implement the `SUGGEST_URI_PATH_QUERY` path to return search suggestions dynamically.

Return rows with `_ID`, `SUGGEST_COLUMN_TEXT_1`, `SUGGEST_COLUMN_INTENT_DATA` columns.

Register with `SearchManager` via a `searchable.xml` configuration in `res/xml`.

`ContentObserver` monitors a URI for changes — ideal for live-updating UI components.

Register observers with `resolver.registerContentObserver(uri, true, observer)`.

Unregister observers in `onStop()` or `onDestroy()` to prevent memory leaks.

Search & Observer

```
// Query with URI-based search filter
val searchUri = Uri.withAppendedPath(
    MyProvider.USERS_URI,
    Uri.encode("yatharth"))

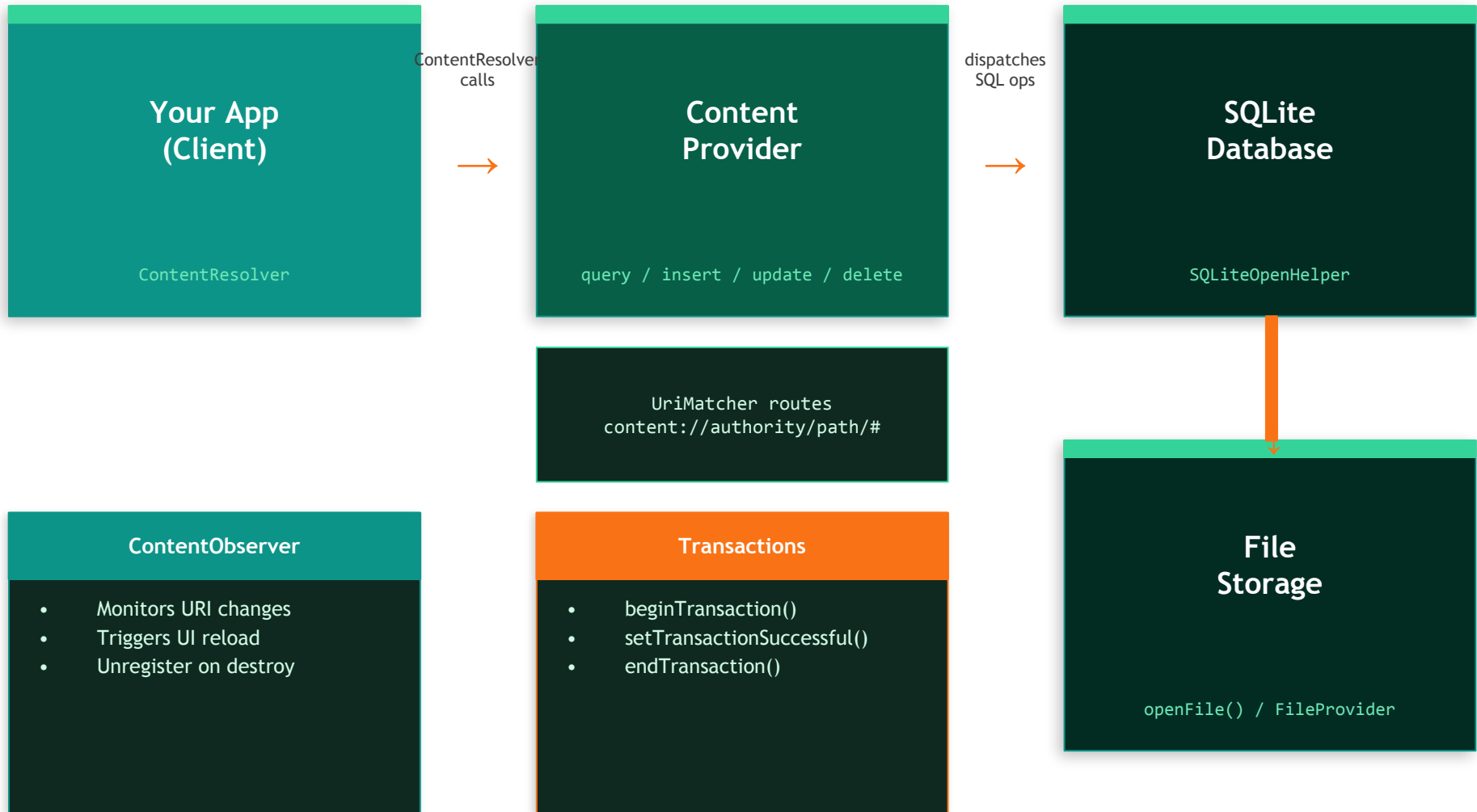
val cursor = resolver.query(
    searchUri,
    arrayOf("_id", "name", "email"),
    null, null, "name ASC")

// ContentObserver – live updates
val observer = object
: ContentObserver(Handler(
    Looper.getMainLooper())) {
    override fun onChange(self: Boolean) {
        super.onChange(self)
        loadUsers() // Refresh UI
    }
}

resolver.registerContentObserver(
    MyProvider.USERS_URI,
    true, // notifyForDescendants
    observer)

// Cleanup
override fun onStop() {
    super.onStop()
    resolver.unregisterContentObserver(observer)
}
```

Architecture: Database ↔ Content Provider ↔ Client App



Key Takeaways

SQLite

Embedded RDBMS — no server, ACID-compliant, SQL standard

Transactions

Atomic multi-step operations using begin/end transaction

SQLiteOpenHelper

Manages creation, versioning and migration of the database

File Storage

openFile() / FileProvider for binary blobs and secure sharing

CRUD Operations

insert/query/update/delete via ContentValues and selectors

ContentObserver

Live change monitoring enabling reactive UI updates

Content Provider

Cross-app data sharing via content:// URIs with permissions

Search

URI-based filtering and suggestion providers for rich search

Topics for Self Study

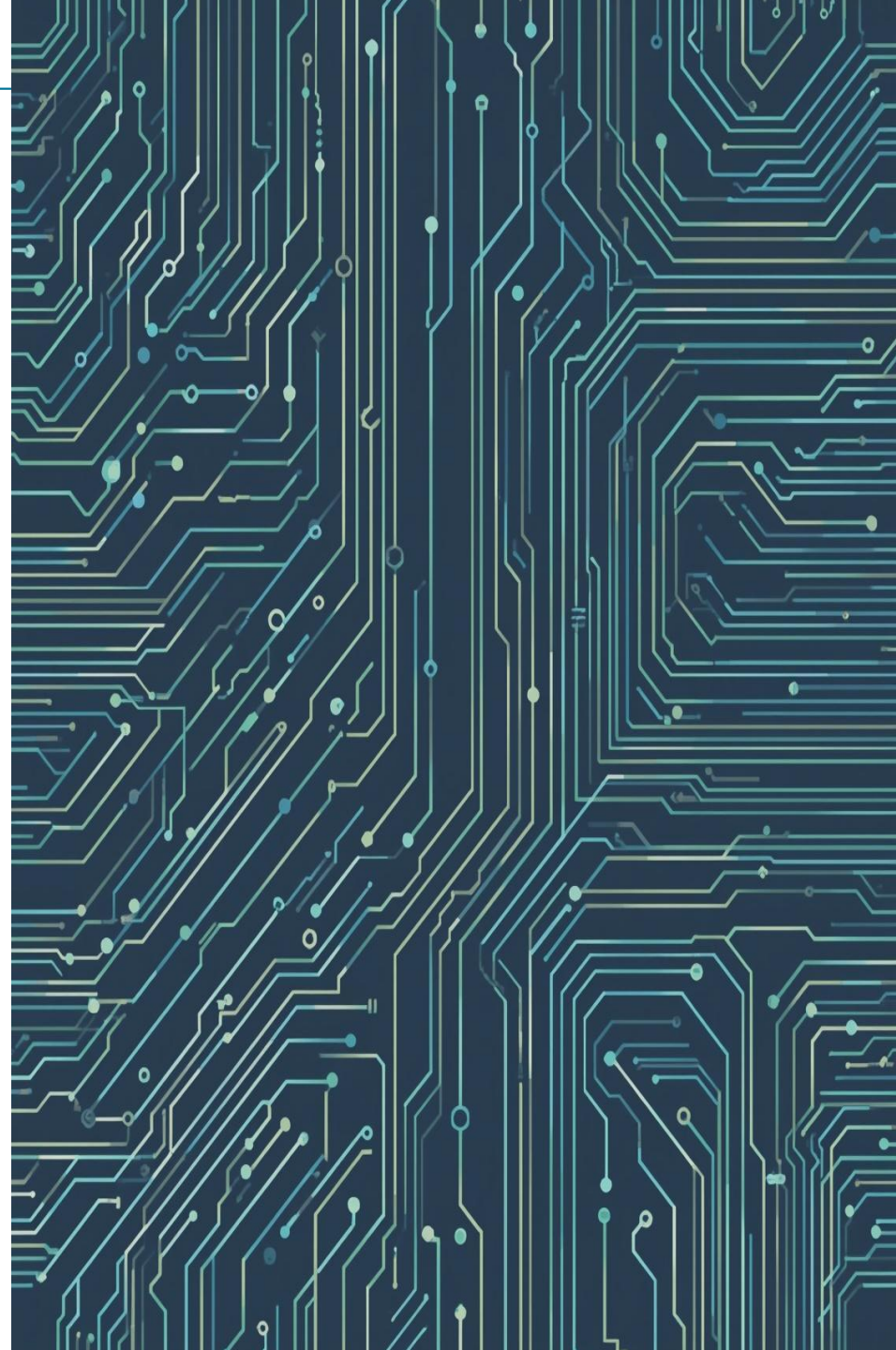
Animation & Activity Navigation

- Animation techniques (transparency, scaling, movement, rotation)
- Smooth UI transitions
- Creating and invoking second activity
- Intent-based navigation

Data Storage & Sharing

- SQLite database (create, insert, update, delete, query)
- Transactions in databases
- Content Providers (data sharing between apps)
- CRUD operations using Content Providers

Databases and Content Providers in Android



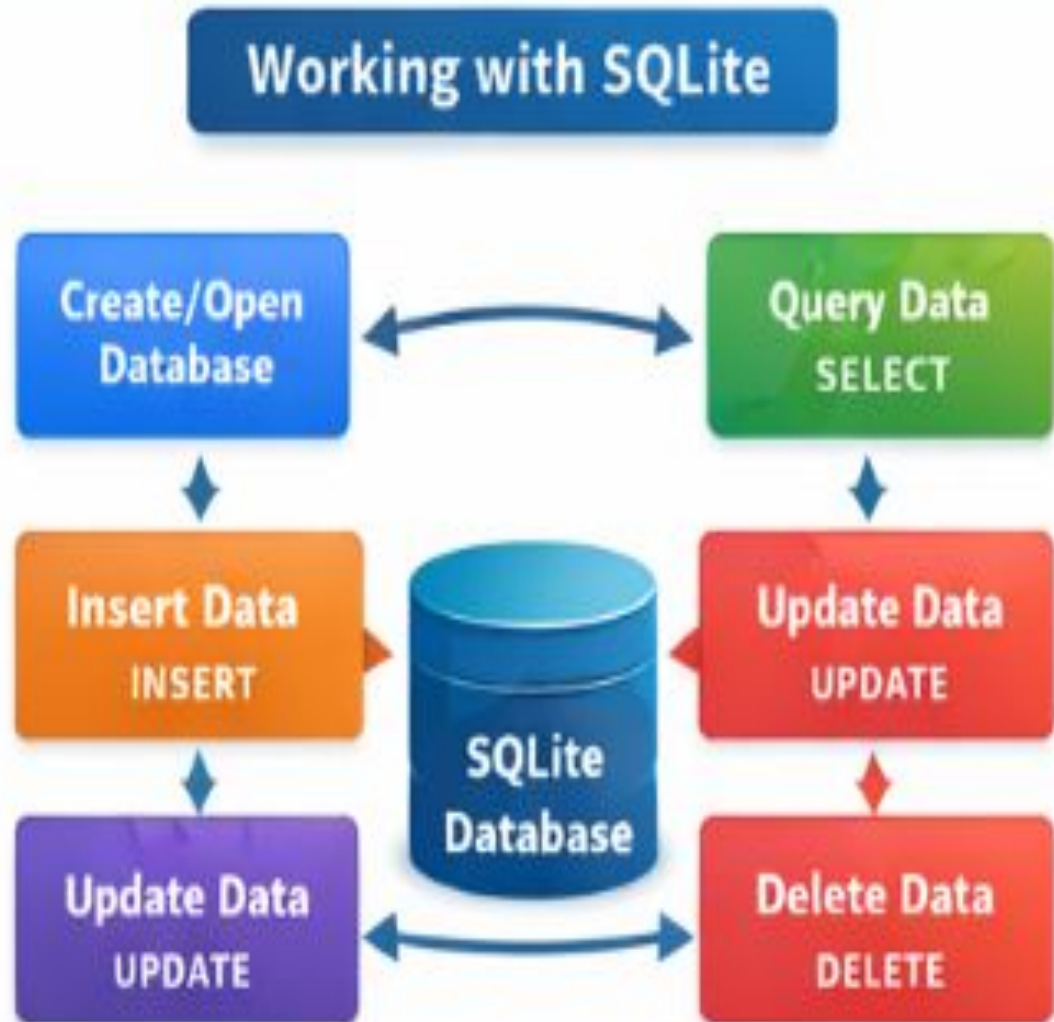
Introduction to SQLite

- SQLite is a lightweight, embedded database in Android
- Used for persistent local storage
- No server required (runs inside app)
- Stores data in tables (rows & columns)
- Uses SQL queries



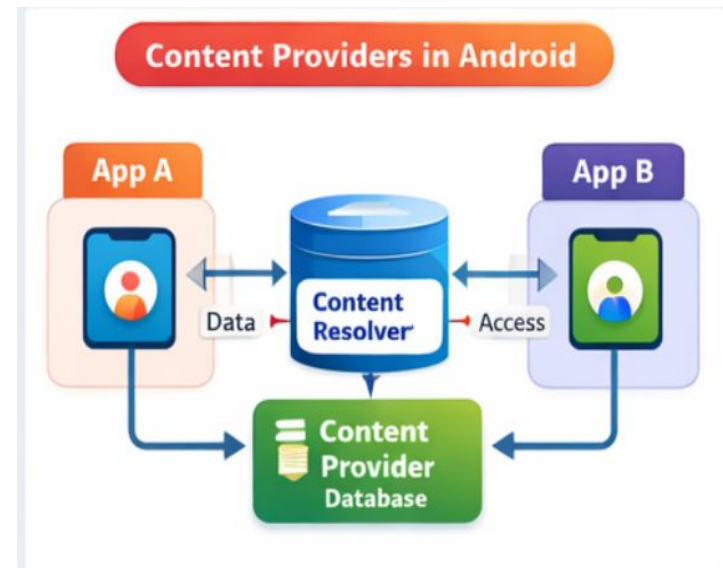
Database Operations

- **Create/Open:** DatabaseSQLiteOpenHelper
- **Query Database:** SELECT statements
- **Insert Data**INSERT: operation
- **Update Data:** UPDATE operation
- **Delete Data:** DELETE operation



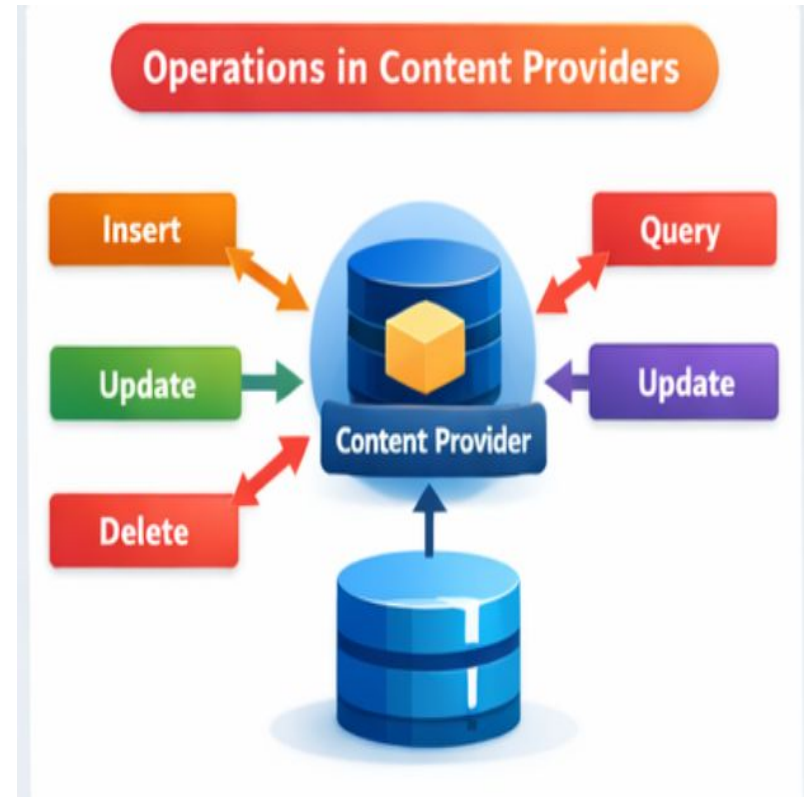
Content Providers

- Used to share data between apps
- Acts as a data abstraction layer
- Data accessed using URI (Uniform Resource Identifier)
- Works with ContentResolver
- Ensures secure data access.



Content Provider Operations

- **Insert** → Add new data
- **Query** → Retrieve data
- **Update** → Modify data
- **Delete** → Remove data
- Internally connected to SQLite database



Transactions & File Storage

Transactions

- Ensure data consistency
- All operations succeed or fail together

File Storage

- Store files using Content Providers
- Secure file sharing between apps



Accessing & Searching Data

- Access using ContentResolver
- Use URI-based queries
- Efficient data retrieval
- Supports search functionality
- Used in apps like Contacts, Media





Thank You!